Class #7

03.02 Quality Attributes & Attribute-Driven Design

Software Architectures · Master in Informatics Engineering

Cláudio Teixeira · claudio@ua.pt

Agenda

- **Block 1 – Making Quality Attributes Measurable** 20 min
  - Candidate boundaries
  - Scenario structure
  - RocketTickets example
- **Block 2 – From Scenarios to Design Pressure** 60 min
  - Prioritization
  - Tactics vs patterns
  - Which quality attributes deserve live focus
  - Mini exercise: Gate validation under partial connectivity
- **Block 3 – Attribute-Driven Design (ADD)** 100 min
  - Driver-first design
  - ADD iteration loop
  - Worked example: RocketTickets
  - Group exercise: Multi-channel ticket distribution and trusted validation

⏱ 0:00 – 20 min

Block 1

Making Quality Attributes Measurable

"Good quality" depends on who is feeling the pain

The same system can feel "good" or "bad" for completely different reasons.

Stakeholder	What quality feels like to them
Customer	"I can find and buy a seat quickly."
Support team	"We can explain what happened when a purchase fails."
Finance	"Payments and refunds reconcile correctly."
Delivery team	"We can change one part without breaking the rest."
Security / compliance	"Sensitive data is not exposed or altered."

Quality starts as perception. Architecture work begins when we turn that perception into something testable.

Start with candidate boundaries

Before discussing quality, decide **which part of the system** each concern belongs to.

Candidate boundary	Why we separate it	What we will ask about it next
SeatAvailability	answers what can still be sold right now	can it stay responsive under spike traffic?
Reservation	protects holds, confirmations, and seat-state rules	can it prevent double sale and timeout errors?
Ticketing	issues and delivers confirmed tickets	what can be delayed safely after payment succeeds?
Payments	external concern with unstable latency and failure modes	what happens when the provider is slow or fails?

These are not final architecture decisions yet. They are working boundaries so the next slides can turn vague concerns into measurable scenarios.

Ticket drop night

RocketTickets opens sales for a stadium show at `10:00`.

- 50,000 seats
- 20x traffic spike in the first minute
- payment provider latency is unstable
- support cannot explain oversold VIP seats away afterward

Different failures will trigger different architectures.

WHICH FAILURE WOULD YOU LEAST ACCEPT?

Slow seat search, oversold seats, payment exposure, or a deployment freeze during the campaign?

"The system must be fast" is not a requirement

That sentence is too weak to drive architecture.

Missing piece	Why architecture cannot answer it
Who triggers it	Load shape changes the solution
What exactly happens	Search, checkout, and settlement are different problems
Where it lands	Whole system is too imprecise
In what conditions	Normal traffic and on-sale spikes are different worlds
How good is good enough	No measure means no design target

Quality attribute scenarios make quality testable

Use one common shape for every quality discussion.

Part	Question it answers
Stimulus source	Who or what causes the event?
Stimulus	What arrives?
Artifact	What part of the system is hit?
Environment	Under what conditions?
Response	What must the system do?
Response measure	How will we know it succeeded?

Example scenario — performance under launch pressure

Part	RocketTickets scenario
Stimulus source	20,000 customers and the mobile app
Stimulus	Seat-availability requests for the same event
Artifact	`SeatAvailability` context
Environment	First 60 seconds of a public on-sale
Response	Return available sections without blocking checkout writes
Response measure	`p95 < 1.5s`, freshness under `2s`

Now the architect has something to design against.

Example scenario — availability under provider failure

Part	RocketTickets scenario
Stimulus source	external payment provider
Stimulus	timeout or failed charge confirmation
Artifact	`Reservation` boundary
Environment	sale is in progress and active holds already exist
Response	preserve seat-state consistency, avoid double sale, and release expired holds safely
Response measure	no oversold seats, hold recovery under `5 min` , failed payment visible to support within `30s`

The same structure works for runtime failure, not only for speed.

reference

Revisit

Block 1 — Reference

> *Full scenario vocabulary, full structure, and scenario-writing guidance for quality attributes.*

Revisit — Terminology

Quality attribute — a measurable property that describes how well the system behaves under certain conditions, beyond basic functional correctness.

Scenario — a short, testable description of how the system should react to a stimulus.

Artifact — the specific part of the system being affected, not just "the system".

Environment — the operating condition in which the event happens: normal load, degraded mode, release window, flash sale, and so on.

Response measure — the metric that decides whether the scenario was met.

Revisit — The Six-Part Structure

Part	Meaning	Typical mistake
Stimulus source	who or what originates the event	omitted completely
Stimulus	what arrives or happens	described too vaguely
Artifact	what part is hit	saying only "the platform"
Environment	what operating state applies	assuming normal load by default
Response	what the system must do	confusing it with the measure
Response measure	how success is verified	using words like fast or secure

Revisit — General Scenario Template

Use this sentence frame when the discussion gets vague:

> When **stimulus source** causes **stimulus** on **artifact** under **environment**, the system should **response** within **response measure**.

Example:

> When 20,000 customers request seat availability on `SeatAvailability` during the first minute of a public on-sale, the system should return results without blocking writes and keep `p95` latency under `1.5s`.

Revisit – Performance Scenarios

Performance is about time, throughput, and resource use under real load.

Sub-question	Typical measure
How fast?	`p95`, `p99`, average latency
How much?	requests/sec, events/sec
Under what load?	concurrent users, burst rate, message volume
With what resources?	CPU, memory, DB utilization

Typical tactics: caching, concurrency, partitioning, asynchronous processing, load balancing.

Revisit — Availability Scenarios

Availability is the ability to remain useful despite failure.

Scenario concern	Typical measure
Survive service failure	uptime %, failover time
Recover data path	MTTR
Avoid cascading failure	% successful degraded requests
Preserve essential actions	checkout success during partial outage

Typical tactics: redundancy, failover, health checks, circuit breakers, bulkheads.

Revisit — Security Scenarios

Security scenarios should state exactly what must be protected.

Concern	Question
Confidentiality	who must not read the data?
Integrity	who must not alter the data?
Availability	who must not deny the service?

Typical tactics: authentication, authorization, encryption, audit trails, rate limits, secret isolation.

Bad: "the system must be secure"

Better: "an unauthenticated client cannot retrieve another user's invoice data, and all such attempts are logged within 5 seconds."

Revisit — Modifiability Scenarios

Modifiability is a change-cost question.

Good scenario element	Example
Change trigger	add a payment provider
Artifact	payment integration boundary
Environment	during active development
Response	implement without changing checkout core
Measure	one team, 2 days, no regression outside payment tests

This is the QA most often left vague, even though it dominates lifetime cost.

Revisit — Common Scenario Writing Failures

- choosing the whole system as the artifact every time
- omitting the environment
- using adjectives instead of measures
- writing architecture decisions instead of scenarios
- mixing multiple quality attributes into one sentence

If a scenario already says "use microservices", it is no longer a requirement. It is already a solution.

Revisit — References

Bibliography

Bass, L., Clements, P., Kazman, R.

Software Architecture in Practice (4th ed.). Addison-Wesley.

Primary reference for quality attribute scenarios, tactics, and architecture trade-offs.

Ingeno, J.

Software Architect's Handbook. Packt.

Referenced in the original material for quality attributes and design guidance.

O'Reilly section 1

O'Reilly section 2

Additional reading

Ford, N., Richards, M., Sadalage, P., Dehghani, Z.

Software Architecture: The Hard Parts. O'Reilly Media.

Referenced in the original material for trade-offs and architectural thinking.

O'Reilly chapter 1

Original source material

Quality attributes source pack.

⏱ 0:20 – 60 min

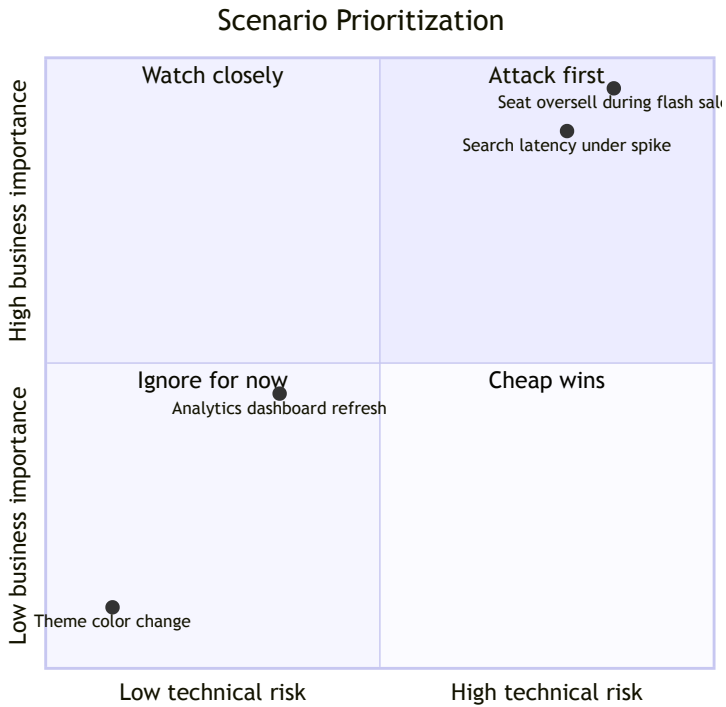
Block 2

From Scenarios to Design Pressure

Not every scenario deserves the same priority

Prioritize with two axes:

- **Business importance**: how much the outcome matters
- **Technical risk**: how likely the current design is to fail



Select concepts because they answer a driver

PATTERN EXAMPLES

Layered architecture
Changes the macro structure.

Microservices
Changes boundaries and deployment units.

TACTIC EXAMPLES

Caching
Changes response time and load shape.

Retry with backoff
Changes failure behavior.

Circuit breaker
Changes dependency containment.

Encryption at rest
Changes exposure if storage is breached.

> Patterns organize the system. Tactics target a scenario.

The catalogue is broader than the core path here

This walkthrough focuses on the attributes that drive the strongest design moves first. The broader catalogue, tactics, and trade-offs are captured in the Block 2 reference section and bibliography.

Core focus here	Expanded in reference section
Performance	Deployability
Availability	Integrability
Security	Energy efficiency
Modifiability	Portability / safety variants

Use the reference section when the design problem shifts, not as a checklist to exhaust.

Four quality attributes worth prioritizing here

Attribute	What the architect is really asking
Performance	Will the system stay responsive under the load that matters?
Availability	What failures can users survive without the service disappearing?
Security	What must never be exposed, forged, or escalated?
Modifiability	How expensive will the next change be, and where will it spread?

Everything else can be framed with the same scenario structure later.

Mini Exercise

Gate Validation Under Partial Connectivity

RocketTickets must validate entry at stadium gates using fixed and handheld scanners. Tickets may come from RocketTickets directly or from authorized resale partners. Connectivity at the venue is unstable, but duplicate entry and forged tickets are unacceptable.

Exercise document →

docs.google.com/document/d/1MeahYzHH1IhKI9Q4v-tAzLiDxs_mSRn6aQKJkG2V83s

20'
DURATION

3-4
PER GROUP

What the gate validation case should teach you

- Offline operation is not the same thing as relaxed control.
- Trust, latency, and reconciliation pull the design in different directions.
- A validation device is part of the architecture, not just hardware at the edge.
- Diagrams, tables, and decision notes matter only if they justify a concrete position.

reference

Revisit

Block 2 — Reference

> *Deeper quality-attribute catalogue, trade-offs, and tactic vocabulary that supports Block 2.*

Revisit — Patterns vs Tactics

Dimension	Pattern	Tactic
Scope	macro structure	targeted quality move
Examples	layered, microservices, event-driven	cache, retry, circuit breaker, encryption
Question answered	how is the system organized?	how does this quality improve?
Risk if misused	architecture chosen for fashion	local optimization without structural fit

Patterns create the playing field. Tactics alter how the system behaves on that field.

Revisit — Performance Tactics

Tactic	What it improves	Trade-off introduced
Caching	latency, throughput	staleness
Load balancing	throughput, availability	coordination overhead
Asynchronous processing	responsiveness	eventual consistency
Partitioning	scale-out capacity	operational complexity
Resource pooling	efficiency	contention and tuning complexity

The point is not to memorize the list. The point is to match one tactic to one scenario pressure.

Revisit — Availability Tactics

Tactic	Gain	Cost
Active redundancy	near-zero failover time	high cost and sync complexity
Passive redundancy	lower infrastructure cost	slower recovery
Health probes	faster failure detection	false positives if mis-tuned
Circuit breakers	contain cascading failure	recovery behavior becomes more complex
Bulkheads	isolate blast radius	duplicated capacity and complexity

Availability is rarely free. Every extra nine buys resilience by spending complexity.

Revisit — Deployability

Deployability is the ease and safety of changing the running system.

Useful measure	Example
deployment frequency	several times per day
success rate	99% successful releases
rollback time	under 10 minutes
blast radius	one service, not the whole platform

Typical tactics: automated pipelines, feature flags, canary release, health-based rollback.

Revisit – Integrability

Integrability measures how hard it is to connect to other systems without polluting your own model.

Distance type	Example
Syntactic	integer vs string IDs
Semantic	gross price vs net price
Temporal	async webhook vs synchronous expectation
Behavioral	same status name, different rules
Resource	shared limits that do not align

Typical tactics: adapters, facades, anti-corruption layers, API gateways, versioned contracts.

Revisit — Energy Efficiency

Energy efficiency matters most when compute cost, battery, or device footprint are constrained.

Question	Example
can we do less work?	reduce polling frequency
can we wake less often?	batch device communication
can we use fewer resources?	right-size background workers
can we measure the waste?	track CPU time and idle consumption

It is often a hidden cost problem disguised as an infrastructure problem.

Revisit – Portability

Portability is the ability to move the system across environments without a rewrite.

Driver	Architectural consequence
cloud-vendor exit	isolate provider-specific services
compliance relocation	externalize infrastructure assumptions
cost shift	minimize hard dependency on managed vendor features

Typical tactics: configuration isolation, standards-based interfaces, abstraction over provider APIs, containerized runtime assumptions.

Revisit — Trade-off Reminders

If you optimize for…	expect tension with…
performance	consistency
availability	simplicity
security	latency and operability
modifiability	local performance
deployability	strong runtime coupling
portability	deep vendor optimization

The architect’s task is not to remove trade-offs. It is to choose them deliberately and document why.

Revisit — References

Bibliography

Bass, L., Clements, P., Kazman, R. *Software Architecture in Practice* (4th ed.). Addison-Wesley. O'Reilly

Ingeno, J. *Software Architect's Handbook*. Packt. Referenced in the original material for quality attributes, tactics, and broader architecture guidance. O'Reilly section 1 · O'Reilly section 2

Ford, N., Richards, M., Sadalage, P., Dehghani, Z. *Software Architecture: The Hard Parts*. O'Reilly Media. O'Reilly chapter 1

Original source material

Quality attributes source pack for the broader catalogue.

Portability briefing for the portability branch referenced here.

⏱ 1:20 – 100 min

Block 3

Attribute-Driven Design (ADD)

Pattern-first design sounds decisive and usually is not

> "Let's do microservices" is not an architectural driver.

Pattern-first

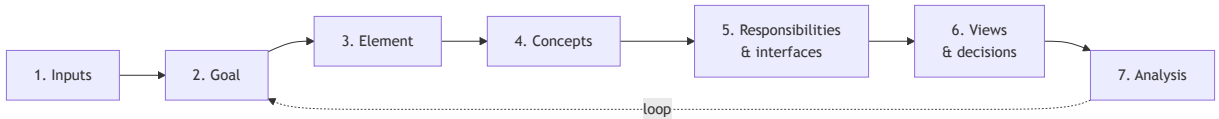
- pick a style early
- justify it afterward
- discover trade-offs too late
- struggle to explain why some complexity exists

Driver-first

- start from scenarios and constraints
- pick the hardest iteration goal
- decompose only where pressure exists
- document why each decision exists

ADD in one sentence

Attribute-Driven Design is an iterative way to decompose a system around the drivers that matter most now.



Frame the iteration

Inputs and goal define what pressure matters now.

Design the response

Refine one element, select concepts, assign responsibilities, and sketch decisions.

Check and loop

Analysis decides whether the iteration stands or sends the design back around.

ADD example – first iteration

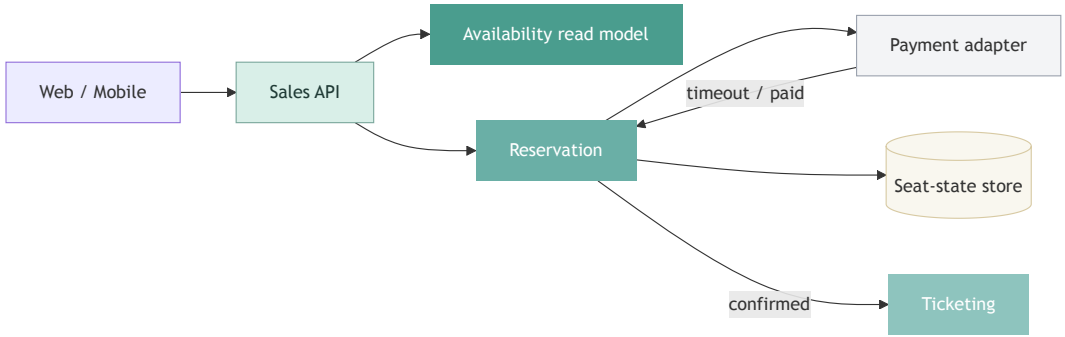
RocketTickets starts with one hard problem:

> keep seat-state correct during flash-sale traffic while payments are slow and unreliable

Input type	Example
Business goal	protect trust during major on-sale events
Scenario A	no oversold seats during payment retries
Scenario B	seat search stays responsive during 20x traffic spikes
Constraint	payment provider is external and unstable

The first iteration goal becomes: protect `Reservation` consistency without collapsing `SeatAvailability`.

ADD example – first-cut response



ADD step	Result
Element refined	`Reservation` boundary
Concepts selected	read model, single seat-state authority, async payment handling
Key decision	separate read pressure from consistency pressure

ADD example — analysis and next loop

Driver	What improved	What remains open
No double sale	one authoritative reservation path	timeout and idempotency rules still need detail
Responsive seat search	reads move away from transactional writes	freshness lag must be bounded
Payment instability	provider latency no longer blocks every step	pending-payment UX is unresolved

That analysis sets up the next iteration:

- tighten timeout and recovery rules
- define freshness tolerance for availability
- decide what support must see during pending payment states

Why ADD starts after QA scenarios

QA work gives you…	ADD needs it for…
Prioritized scenarios	choosing the iteration goal
Named artifacts	deciding what element to refine
Constraints	rejecting attractive but wrong concepts
Response measures	checking whether the iteration actually worked

Without the QA work, ADD becomes architecture theatre.

First iteration inputs — RocketTickets

Assume the architecture begins with these candidate boundaries:

- `SeatAvailability`
- `Reservation`
- `Ticketing`
- external `Payments`

Now assume these are the strongest drivers:

Type	Input
Business goal	Survive flash-sale launches without damaging trust
Scenario A	No oversold seats during payment retries
Scenario B	Seat search remains responsive during 20x traffic spikes
Constraint	External payment provider remains outside our control
Constraint	Sales team needs independent deployment from ticket delivery

That is enough to start an ADD iteration.

Pick one iteration goal, not five

For the first pass:

> Protect seat-state consistency while keeping purchase flow responsive under launch traffic.

This immediately narrows the design space.

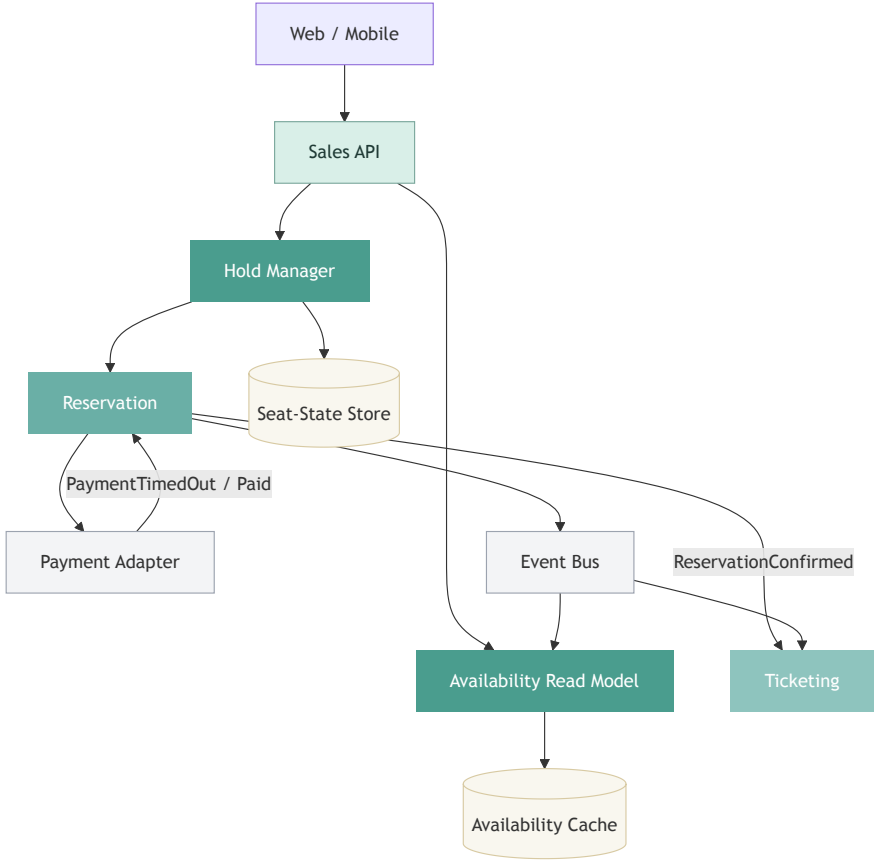
Focus now	Not the focus yet
Sales boundary	Recommendation engine
Seat hold lifecycle	Marketing campaigns
Payment timeout behavior	BI dashboard richness
Interface between checkout and ticketing	Back-office reporting

Select concepts because they answer a driver

Driver	Possible concept or tactic	Why it fits
Read spike on seat lookup	Read-optimized availability model + cache	Reduces contention on transactional writes
No double sale	Single authoritative hold/reservation path	Keeps one seat-state truth
Unstable payment latency	Timeout handling + asynchronous compensation	Prevents payment delay from freezing inventory
Independent deployment	Separate ticketing boundary	Keeps fulfillment change away from seat-state rules

ADD is not picking favorite tools. It is matching design moves to explicit pressure.

One possible first-cut architecture



This is not "the architecture". It is one iteration result that can now be challenged.

Analyze the design against the drivers

Scenario or concern	What improved	What is still risky
Seat search under spike	Read model and cache absorb most read pressure	Freshness lag must stay bounded
No oversell during retries	Single hold / reservation path centralizes seat truth	Timeout and idempotency rules must be precise
Independent ticketing change	Fulfillment moved behind an event boundary	Event contracts can still drift
Payment instability	Checkout can react asynchronously	Customer experience during pending state must be designed

Analysis is where ADD earns its keep.

What ADD does not solve for you

It does **not** replace:

- stakeholder discovery
- broad business framing
- architecture evaluation
- serious documentation discipline

ADD is strong at focused decomposition. It is weak as a full lifecycle by itself.

Exercise

Multi-Channel Ticket Distribution and Trusted Validation

RocketTickets sells through its own channel, partner promoters, and authorized resellers. Every ticket must still be trusted at the gate, even after transfer, resale, revocation, or partner-side delay.

Exercise document →

docs.google.com/document/d/1qDOjuj1CwB2FZho_vd3TE5JDIYX5Wcbi2jw5-L6susk

30'
DURATION

4
PER GROUP

What one ADD pass must make explicit

- the drivers that justify the iteration
- the design choices made in response
- the trade-off that remains consciously accepted

That is enough to decide whether to loop on the same pressure or move to another attribute.

reference

Revisit

Block 3 — Reference

> *Full ADD step detail, iteration outputs, and common failure modes.*

Revisit — What ADD Is and Is Not

ADD is a design method for focused architectural decomposition.

It is **good at**:

- turning prioritized drivers into concrete structure
- decomposing one part of a system at a time
- recording why a design decision exists

It is **not enough for**:

- enterprise-wide governance
- stakeholder discovery from scratch
- complete architecture documentation
- architecture evaluation by itself

Revisit – The Iteration Steps

Step	Question
1. Review inputs	what drivers, constraints, and requirements do we have?
2. Pick iteration goal	what problem are we solving now?
3. Choose element to refine	where will we decompose?
4. Select design concepts	what patterns, tactics, or external services fit?
5. Instantiate elements	what responsibilities and interfaces appear?
6. Sketch views and decisions	how will we communicate the result?
7. Analyze current design	did the iteration satisfy its drivers?

This loop repeats until the architecture is decomposed enough for the next level of design.

Revisit — Inputs Worth Keeping Explicit

Input type	Example
business goals	grow to 1M users without losing trust
functional requirements	real-time device monitoring
quality scenarios	p95 under 1 second during burst load
constraints	external identity provider is mandatory
architectural concerns	independent deployment, auditability
existing architecture	legacy monolith, event bus already present

The method gets weak the moment these inputs remain implicit.

Revisit — Good Iteration Goals

Bad iteration goal:

> Design the architecture.

Good iteration goal:

> Keep seat-state consistency during payment retries while preserving responsive seat search during flash-sale traffic.

Good goals are narrow, driver-linked, and testable against scenarios.

Revisit — Design Concepts

The "design concepts" step can draw from multiple sources:

- architecture patterns
- tactics
- reference architectures
- externally provided components
- organizational standards

A concept is chosen because it helps satisfy a driver, not because it is fashionable or already familiar to the team.

Revisit — What to Produce Per Iteration

At minimum, each ADD iteration should leave behind:

- one short statement of the iteration goal
- the drivers used in that iteration
- a sketch of the refined structure
- responsibilities for the refined elements
- the exposed interfaces
- 2-5 decisions with rationale
- one short analysis against the scenarios

If these artifacts do not exist, the iteration happened only in conversation.

Revisit — Common ADD Failures

- choosing too many drivers in one iteration
- decomposing the whole system at once
- selecting concepts before naming the goal
- documenting the final diagram but not the reasoning
- calling the iteration done without checking response measures

The most common mistake is scale: teams try to finish the whole architecture in the first iteration.

Revisit — SHEMS Example Framing

For SHEMS, a strong first iteration might focus on:

Candidate goal	Why it is a good first pass
real-time monitoring responsiveness	pressure is easy to visualize and measure
device-control reliability	forces a clear command path and failure policy
privacy and consent boundaries	forces separation of sensitive data responsibilities

Weak first-pass goals are broad bundles like:

> scalability, usability, security, AI adaptation, and maintainability

That is not an iteration. That is the entire project.

Revisit — References

Bibliography

Bass, L., Clements, P., Kazman, R. *Software Architecture in Practice* (4th ed.). Addison-Wesley. O'Reilly

Ingeno, J. *Software Architect's Handbook*. Packt. Referenced in the original material for the ADD method and architecture design guidance. O'Reilly section 1 · O'Reilly section 2

Ford, N., Richards, M., Sadalage, P., Dehghani, Z. *Software Architecture: The Hard Parts*. O'Reilly Media. O'Reilly chapter 1

Original source material

ADD source pack for the method and the SHEMS framing.

Smart Home Energy Management System exercise brief.